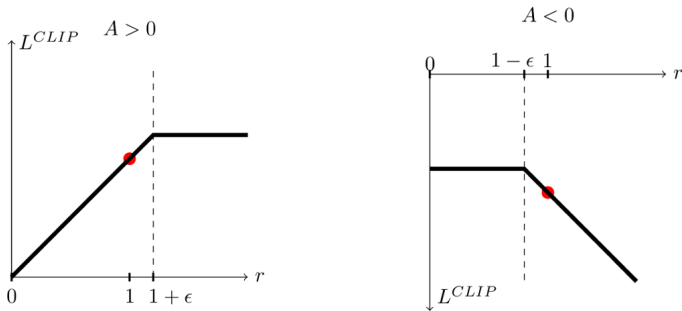


- 问题：在强化学习中，直接优化策略会导致不稳定的训练，模型可能因为过大的参数更新而崩溃。
- 解决方案：PPO通过限制策略更新幅度，使得每一步训练都不会偏离当前策略太多，同时高效利用采样数据。

2. PPO 的核心思想

PPO 的目标是通过以下方式改进策略梯度优化：

1. 限制策略更新幅度，防止策略过度偏离。
2. 使用优势函数 $A(s, a)$ 来评价某个动作的相对好坏。



优化目标

PPO 的目标函数如下：

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

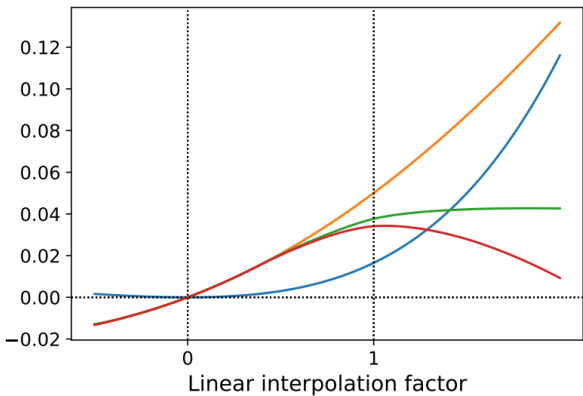
其中：

$r_t(\theta)$: 概率比率 $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ 它表示新策略和旧策略在同一状态下选择动作的概率比值。

A_t : 优势函数 通过以下公式计算： $A_t = Q(s_t, a_t) - V(s_t)$ 或者用广义优势估计（GAE）的方法近似。

剪辑操作 $\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)$: 将 $r_t(\theta)$ 限制在区间 $[1 - \epsilon, 1 + \epsilon]$ 内，防止策略变化过大。

r_t



3. 为什么 PPO 很强？

1. 简洁性：比 TRPO（Trust Region Policy Optimization）更简单，无需二次优化。
2. 稳定性：使用剪辑机制防止策略更新过度。
3. 高效性：利用采样数据多次训练，提高样本利用率。

4. PPO 的直观类比

假设你是一个篮球教练，训练球员投篮：

- 如果每次训练完全改变投篮动作，球员可能会表现失常（类似于策略更新过度）。
- 如果每次训练动作变化太小，可能很难进步（类似于更新不足）。
- PPO 的剪辑机制就像一个“适度改进”的规则，告诉球员在合理范围内调整投篮动作，同时评估每次投篮的表现是否优于平均水平。

PPO算法的流程推导及数学公式

PPO（Proximal Policy Optimization）也是一种策略优化算法，它的核心思想是对策略更新进行限制，使训练更加稳定，同时保持效率。以下是其数学公式和整体流程：

1. 算法目标

强化学习的核心目标是优化策略 π_{θ} ，最大化累积奖励 R 。策略梯度方法（如REINFORCE）直接优化策略，但更新过大可能导致不稳定。为了解决这个问题，PPO通过引入限制更新幅度的机制，保证策略的稳定性。

目标是优化以下期望： $J(\theta) = \mathbb{E} \pi_{\theta} [R]$

通过梯度上升法更新策略。

2. PPO的概率比率

PPO在优化过程中引入了**概率比率**，用于衡量新旧策略的差异：

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

- $\pi_{\theta}(a_t|s_t)$: 新策略对动作 a_t 的概率。
- $\pi_{\theta_{\text{old}}}(a_t|s_t)$: 旧策略对动作 a_t 的概率。

这个比率表示策略变化的程度。

3. 优化目标

为了限制策略的更新幅度，PPO引入了剪辑目标函数：

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

PPO的目标是找到一个折中：在保持改进的同时防止策略变化过大。

4. 值函数优化

PPO不仅优化策略，还同时更新值函数 $V(s_t)$ ，通过最小化均方误差来更新：

$$L^{VF}(\theta) = \mathbb{E}_t [(V(s_t; \theta) - R_t)^2]$$

- $V(s_t; \theta)$: 当前状态的值函数预测。
- $R_t = \sum_{k=0}^n \gamma^k r_{t+k}$: 累计回报。

这个**损失函数** 使得Critic能够更准确地估计状态值。

5. 策略熵正则化

为了鼓励策略的探索，PPO引入了熵正则化项：

$$L^{ENT}(\theta) = \mathbb{E}_t [H(\pi_{\theta}(s_t))]$$

- $H(\pi_{\theta}(s_t))$: 策略的熵，表示策略分布的不确定性。
- 增加熵可以防止策略过早收敛到局部最优。

6. 总损失函数

PPO结合策略损失、值函数损失和熵正则化项，形成总损失函数：

$$L(\theta) = \mathbb{E}_t \left[L^{CLIP}(\theta) - c_1 L^{VF}(\theta) + c_2 L^{ENT}(\theta) \right]$$

- c_1 和 c_2 ：权重系数，用于平衡策略优化、值函数更新和熵正则化。

7. PPO 算法流程

PPO 可以简化为以下步骤：

采样：使用当前策略 π_θ 与环境交互，收集状态 s_t 、动作 a_t 、奖励 r_t 。

计算优势函数：

评估某个动作 a_t 在状态 s_t 下相对于平均表现的优劣（优势函数 A_t ）。
利用 A_t 引导策略改进。

计算概率比率 $r_t(\theta)$ ：比较新策略和旧策略对动作 a_t 的选择概率。

策略更新：

如果更新过大（超出剪辑范围 $1 - \epsilon$ 到 $1 + \epsilon$ ），会被惩罚。
保证更新幅度适中，既不太保守，也不太激进。

值函数更新：用以下损失函数优化值函数 $V(s_t)$: $L^{VF}(\theta) = \mathbb{E}_t \left[(V(s_t) - R_t)^2 \right]$

重复以上步骤：通过多轮迭代，使策略逐步优化，直到收敛。

8.PPO算法的关键

PPO的关键公式和目标可以概括如下：

- 核心目标：优化策略，使 $r_t(\theta)A_t$ 的改进在限制范围内。
- 限制更新幅度：通过剪辑函数 $\text{clip}()$ ，避免策略更新过大导致不稳定。
- 同时优化值函数：通过 $L^{VF}(\theta)$ ，提高Critic的预测精度。
- 探索与稳定性平衡：通过 $L^{ENT}(\theta)$ ，鼓励策略探索。

这种设计使得PPO在训练过程中高效且稳定，是目前强化学习中的常用算法之一。

[Python] PPO算法的代码实现

以下是使用 **PyTorch** 实现 PPO（Proximal Policy Optimization）算法的完整代码

项目代码我已经放入[GitHub](#)里面，可以通过下面链接跳转：🔥

[【强化学习】---PPO算法](#)

后续相关单智能体强化学习算法也会不断在 **【强化学习】** 项目里更新，如果该项目对你有所帮助，请帮我点一个[星星](#)🌟🌟🌟🌟🌟，鼓励分享，十分感谢!!!

若是下面代码复现困难或者有问题，也欢迎[评论区留言](#)。

python

运行本项目复制

```
1 """ 《PPO算法的代码》
2     时间：2024.12
3     环境：gym
4     作者：不去幼儿园
5 """
6 import torch # Import PyTorch, a popular machine learning library
7 import torch.nn as nn # Import the neural network module
8 import torch.optim as optim # Import optimization algorithms
```



逐行解释 PPO 代码和公式

以下是对实现的 **PyTorch PPO 算法代码** 的详细解释，逐行结合公式解析：

1. Actor-Critic 神经网络

python

运行本项目

复制

```
1 # Define Actor-Critic Network
2 class ActorCritic(nn.Module): # Define the Actor-Critic model
3     def __init__(self, state_dim, action_dim): # Initialize with state and action dimensions
4         super(ActorCritic, self).__init__() # Call parent class constructor
5         self.shared_layer = nn.Sequential( # Shared network layers for feature extraction
6             nn.Linear(state_dim, 128), # Fully connected layer with 128 neurons
7             nn.ReLU() # ReLU activation function
8         )
9         self.actor = nn.Sequential( # Define the actor (policy) network
```

- **shared_layer**: 将状态 s 映射到一个隐层表示，使用 ReLU 激活函数。
- **actor**: 输出策略 $\pi_{\theta}(a|s)$ ，表示在状态 s 下选择动作 a 的概率分布。使用 Softmax 确保输出是概率。
- **critic**: 输出状态值函数 $V(s)$ ，表示在状态 s 下的预期累计奖励。

$$\pi_{\theta}(a|s) = \text{Softmax}(W_a \cdot f(s)) [V(s) = W_v \cdot f(s)]$$

其中 $f(s)$ 是共享层的输出， W_a 和 W_v 是权重矩阵。

2. Memory 类

python

运行本项目

复制

```
1 # Memory to store experiences
2 class Memory: # Class to store agent's experience
3     def __init__(self): # Initialize memory
4         self.states = [] # List to store states
5         self.actions = [] # List to store actions
6         self.logprobs = [] # List to store log probabilities of actions
7         self.rewards = [] # List to store rewards
8         self.is_terminals = [] # List to store terminal state flags
```

用于存储一个 episode 的经验数据：

states: 状态、actions: 动作、logprobs: 动作的对数概率、rewards: 即时奖励、is_terminals: 是否为终止状态（布尔值）

作用：为后续策略更新提供样本数据。

3. PPO 初始化

python

运行本项目

复制

```
1 # PPO Agent
2 class PPO: # Define the PPO agent
3     def __init__(self, state_dim, action_dim, lr=0.002, gamma=0.99, eps_clip=0.2, K_epochs=4):
4         self.policy = ActorCritic(state_dim, action_dim).to(device) # Initialize the Actor-Critic model
5         self.optimizer = optim.Adam(self.policy.parameters(), lr=lr) # Adam optimizer for parameter updates
6         self.policy_old = ActorCritic(state_dim, action_dim).to(device) # Copy of the policy for stability
7         self.policy_old.load_state_dict(self.policy.state_dict()) # Synchronize parameters
8         self.MseLoss = nn.MSELoss() # Mean Squared Error loss for critic updates
```

- **policy**: 当前策略网络，用于输出动作概率和状态值。
- **policy_old**: 旧策略网络，用于计算概率比率 $r_t(\theta)$ 。
- **gamma**: 折扣因子，用于奖励的时间衰减。
- **eps_clip**: 剪辑阈值 ϵ ，用于限制策略更新幅度。

4. 动作选择

python

运行本项目

复制

```
1 def select_action(self, state, memory):
2     state = torch.FloatTensor(state).to(device) # Convert state to PyTorch tensor
3     action_probs, _ = self.policy_old(state) # Get action probabilities from old policy
4     dist = Categorical(action_probs) # Create a categorical distribution
5     action = dist.sample() # Sample an action from the distribution
6
7     memory.states.append(state) # Store state in memory
8     memory.actions.append(action) # Store action in memory
9     # Store log probabilities of the action
```

- **action_probs**: 当前策略下的动作概率 $\pi_{\theta}(a|s)$ 。
- **dist.sample()**: 按照概率分布采样动作 a_t 。
- **log_prob(action)**: 记录动作的对数概率 $\log \pi_{\theta}(a_t|s_t)$ 。

5. 策略更新

python

运行本项目

复制

```
1 def update(self, memory):
2     # Convert memory to tensors
3     old_states = torch.stack(memory.states).to(device).detach() # Convert states to tensor
4     old_actions = torch.stack(memory.actions).to(device).detach() # Convert actions to tensor
5     old_logprobs = torch.stack(memory.logprobs).to(device).detach() # Convert log probabilities to tensor
6
7     # Monte Carlo rewards
8     rewards = [] # Initialize rewards list
9     # Initialize discounted reward
```

6. Surrogate Loss

python

运行本项目

复制

```
1 # Update for K epochs
2 for _ in range(self.K_epochs):
3     # Get action probabilities and state values
4     action_probs, state_values = self.policy(old_states) # Get action probabilities and state values
5     dist = Categorical(action_probs) # Create a categorical distribution
6     new_logprobs = dist.log_prob(old_actions) # Compute new log probabilities of actions
7     entropy = dist.entropy() # Compute entropy for exploration
8
9     # Calculate ratios
```

- 比率: $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{old}(a_t|s_t)}$
- 策略损失: $L^{CLIP}(\theta) = \min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)$

- 价值函数损失: $L^{VF}(\theta) = (V(s_t) - R_t)^2$
- 熵正则化: $H(\pi_\theta(s_t)) = -\sum_a \pi_\theta(a|s_t) \log \pi_\theta(a|s_t)$

7.主程序

python

运行本项目

复制

```
1 # Hyperparameters
2 device = torch.device("cuda" if torch.cuda.is_available() else "cpu") # Use GPU if available
3 env = gym.make("CartPole-v1") # Initialize CartPole environment
4 state_dim = env.observation_space.shape[0] # Dimension of state space
5 action_dim = env.action_space.n # Number of possible actions
6 lr = 0.002 # Learning rate
7 gamma = 0.99 # Discount factor
8 eps_clip = 0.2 # Clipping parameter
9 K_epochs = 4 # Number of epochs for policy update
```

[Notice] 代码解释

Actor-Critic 网络结构:

ActorCritic 模型通过共享层生成动作概率和状态值。

PPO 的优化目标:

使用裁剪的目标函数限制策略更新幅度，避免过度更新。

state_values 通过 Critic 网络提供状态价值的估计。

内存存储:

Memory 用于存储每一轮的状态、动作、奖励和终止标志。

策略更新:

通过多个 epoch 更新，计算 优势函数 和 裁剪后的策略梯度。

奖励归一化:

使用标准化方法对奖励进行处理，以加快收敛。

训练循环:

PPO 从环境中采样，更新策略，打印每一集的总奖励。

bash

运行本项目

复制

```
1 # 环境配置
2 Python 3.11.5
3 torch 2.1.0
4 torchvision 0.16.0
5 gym 0.26.2
```

由于博文主要为了介绍相关算法的原理和应用的方法，缺乏对于实际效果的关注，算法可能在上述环境中的效果不佳或者无法运行，一是算法不适配上环境，二是算法未调参和优化，三是没有呈现完整的代码，四是等等。上述代码用于了解和学习算法足够了，但若是想直接将上面代码应用于实际项目中，还需进行修改。

总结

PPO 的关键是通过限制策略的变化范围（剪辑），让优化更加稳定，同时通过优势函数引导策略改进，充分利用采样数据。这种平衡使得 PPO 成为许多强化任务的默认算法。

更多强化学习文章，请前往：[【强化学习（RL）】专栏](#)

PPO算法、TRPO算法 和 A3C算法对比

以下是 **PPO算法**、**TRPO算法** 和 **A3C算法** 的区别分析：

特性	PPO (Proximal Policy Optimization)	TRPO (Trust Region Policy Optimization)	A3C (Asynchronous Advantage Actor-Critic)
核心思想	使用裁剪的目标函数，限制策略更新幅度，保持稳定性和效率。	限制策略更新的步幅（Trust Region），通过二次约束优化确保稳定性。	通过异步多线程运行环境并行采样和训练，降低方差并加速收敛速度。
优化目标函数	引入剪辑机制	通过KL散度限制策略更新	优化策略梯度
更新方式	同步更新，支持多轮迭代更新样本数据以提高效率。	同步更新，通过优化约束的目标函数严格限制更新步长。	异步更新，多个线程独立采样和更新全局模型。
计算复杂度	低，计算简单，使用裁剪避免复杂的二次优化问题。	高，涉及二次优化问题，计算复杂，资源需求较大。	较低，依赖异步线程并行计算，资源利用率高。
样本利用率	高效，可重复利用采样数据进行多轮梯度更新。	高效，严格优化目标，提升了样本效率。	较低，因为每个线程独立运行，可能导致数据重复和冗余。
实现难度	中等，使用简单的裁剪方法，适合大多数场景。	高，涉及复杂的约束优化和实现细节。	较低，直接异步实现，简单易用。
收敛速度	快，因剪辑机制限制更新幅度，能快速稳定收敛。	慢，因严格的步幅限制，收敛稳定但需要较多训练迭代。	快，因多线程并行采样，能够显著减少训练时间。
稳定性	高，剪辑机制限制过大更新，避免不稳定行为。	高，严格限制更新步幅，保证策略稳定改进。	较低，异步更新可能导致收敛不稳定（如策略冲突）。
应用场景	广泛使用，适合大规模环境或复杂问题。	适合需要极高稳定性的场景，如机器人控制等。	适合资源受限的场景或需要快速实验的任务，如强化学习测试。
优点	简单易实现，收敛快，稳定性高，是主流强化学习算法。	理论支持强，更新步幅严格受控，策略非常稳定。	异步更新高效，能够充分利用多线程资源，加速训练。
缺点	理论支持弱于TRPO，可能过于保守。	实现复杂，计算资源需求高，更新速度慢。	异步更新可能导致训练不稳定，样本利用率较低。
论文来源	Schulman et al., "Proximal Policy Optimization Algorithms" (2017)	Schulman et al., "Trust Region Policy Optimization" (2015)	Mnih et al., "Asynchronous Methods for Deep Reinforcement Learning" (2016)

三种算法的对比总结：

1. **PPO 是 TRPO 的改进版**：PPO 使用简单的剪辑机制代替了 TRPO 的二次优化，显著降低了实现复杂度，同时保持了良好的稳定性和效率。
2. **A3C 的并行化设计**：A3C 的核心是通过多线程异步更新提升效率，但其稳定性略低于 PPO 和 TRPO。
3. **实用性**：PPO 因其简单、稳定、高效的特点，已成为强化学习领域的主流算法；TRPO 更适合需要极高策略稳定性的任务；A3C 在资源受限的场景下表现

博客都是给自己看的笔记，如有误导深表抱歉。文章若有不当和不正确之处，还望理解与指出。由于部分文字、图片等来源于互联网，无法核实真实出处如涉及相关争议，请联系博主删除。如有错误、疑问和侵权，欢迎评论留言联系作者，或者添加VX：**Rainbook_2**，联系作者。🌟

 [\[交流学习/问题答疑\]](#)

 微信名

本文已生成可运行项目
[一键运行](#)

输入博客地址(支持 CSDN、GitHub、Medium等主流内容网站)

生成项目

【深度强化学习】(6) PPO 模型解析，附Pytorch完整代码

近端策略优化算法PPO（proximal policy optimization），具备 Policy Gradient、TRPO 的部分优点，采样数据和使用随机梯度上升方法优化代替目标函数之间交替进行，但 PP

博观而约取，厚积而薄发

与

与

PPO 算法

PPO 是 TRPO 的一种改进算法。它在实现上简化了 TRPO 中的复杂计算，并且它在实验中的性能大多数情况下会比 TRPO 更好，因此目前常被用作一种常用的基准算法。需要

强化学习 PPO算法和代码

强化学习PPO

u012374012的专栏

TuringEvo专栏

i宏的博客

guoguoazgw的博客

强化学习算法笔记【PPO】

PPO算法原理与实践摘要 PPO（Proximal Policy Optimization）是一种基于重要性采样的强化学习算法。通过GAE（Generalized Advantage Estimation）计算优势函数，结合

《第二十三篇》一文搞懂PPO算法：原理、公式、代码实现全覆盖

PPO（Proximal Policy Optimization）是一种用于 强化学习（Reinforcement Learning） 的策略优化算法，尤其适用于训练强化学习代理或大语言模型（LLMs）在 人类偏好反

ppo

Proximal Policy Optimization (PPO) 算法概述 Proximal Policy Optimization (PPO) 是一种用于强化学习中的策略优化方法，旨在通过改进代理的行为来最大化累积奖励。此

关于我们 招贤纳士 商务合作 寻求报道 400-660-0108 kefu@csdn.net 在线客服 工作时间 8:30-22:00

公安备案号11010502030143 京ICP备19004658号 京网文〔2020〕1039-165号 经营性网站备案信息 北京互联网违法和不良信息举报中心 家长监护 网络110报警服务 中国互联网举报中心 Chrome商店下载 账号管理规范 版权与免责声明 版权申诉 出版物许可证 营业执照

©1999-2025北京创新乐知网络技术有限公司



不去幼儿园

博客等级 码龄5年

领域专家: 人工智能技术领域

183 原创

1万+ 点赞

1万+ 收藏

4万+ 粉丝

已关注

私信

更多优质【一键运行】博文

1 基于yoloV7-pose添加任意关键点 + 多类别分类网络修改

2 计算机视觉算法实战——驾驶员危险行为检测技术深度解析（主页有源码）

3 Python复制图像到另一个文件夹（shutil.copy）

4 【skLearn 回归模型】Lasso ---- 选择最佳正则化参数 < 带交叉验证的Las...

5 python 爬虫遇到字体加密解决方案(woff2) ddddocr + 百度ocr 文字识别

查看更多 >

热门文章

一文了解“期刊”、“JCR分区”、“中科院分区”

72081

【强化学习】近端策略优化算法(PPO)万字详解（附代码）

45476

数学建模四大模型、历年国赛题目以及优秀论文

31155

【强化学习】演员评论家Actor-Critic算法(万字长文、附代码)

29054

【RL Latest Tech】安全强化学习（Safe RL）：理论、方法与应用

22539

分类专栏



强化学习（RL）

68篇



单智能体强化学习【...

14篇



多智能体强化学习【...

11篇



强化学习基础【RL ...

14篇

展开全部

上一篇：【强化学习】异步优势Actor-...

下一篇：【强化学习】深度确定性策略梯...

最新评论

- 【MADRL】多智能体双延迟深度确定性...

不去幼儿园: 可前往主页的gitcode仓库下 ...
- 【MADRL】多智能体双延迟深度确定性...

ahahah__: 博主麻烦分享一下代码，谢谢！
2995131126@qq.com
- 【RL Latest Tech】分层强化学习：Optio...

2301_77910969: 博主你好，最近关注到分
层强化学习方面的内容，可以分享一下f ...
- 【启发式算法】RRT*算法详细介绍（Pyt...

不去幼儿园: 没有B站视频
- 【启发式算法】RRT*算法详细介绍（Pyt...

贝_探索版: 博主刚接触路径规划，想部署rr
t*，博主这个有b站链接吗？跟着一步步： ...

大家在看

- 零基础转行网络安全，需要学习多久才能上
手？

VR暴风雪灾害避险模拟系统-冰雪灾害急救
教学VR模拟场景-VR自然灾害学习机

280
- 前端 小程序学习资料之实战（源码）

739
- 别盲目跟风学计算机！ 适配人群 + 2025 就
业前景（含薪资 / 缺口）全说明

STA2001H 【7.2】 样本矩，联合分布，协
方差

431

最新文章

- 【强化学习】区分理解： 时序差分（TD）、
蒙特卡洛（MC）、动态规划（DP）
- 【强化学习】深入理解： PPO（Proximal
Policy Optimization）和 TRPO（Trust
Region Policy Optimization） 的本质区别
- 【启发式算法】Dynamic A*（D*）算法详细
介绍（Python）

2025年 13篇	2024年 94篇
2023年 61篇	2022年 8篇
2021年 7篇	

目录

近端策略优化算法(PPO)详解

PPO算法介绍

- 1. 背景
- 2. PPO 的核心思想
- 3. 为什么 PPO 很强?
- 4. PPO 的直观类比

PPO算法的流程推导及数学公式

- 1. 算法目标
- 2. PPO的概率比率
- 3. 优化目标
- 4. 值函数优化
- 5. 策略熵正则化

收起 ^